Air University

AIR UNIVERSITY

Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: February 19, 2025

Experiment No. 01

Title: Basic Logic Gates and Verification of their Truth Table

Objectives:

The key goals of this experiment are to understand how AND, OR, NOT, NOR, and NAND logic gates function, build circuits using these gates on a breadboard, and verify their truth tables through practical experimentation.

Equipment and Components:

- Breadboard
- Logic Gate ICs: 7408 (AND), 7432 (OR), 7404 (NOT), 7402 (NOR), 7400 (NAND)
- 5V Power Supply
- Connecting Wires
- Multimeter (optional for voltage measurement)
- LEDs with suitable resistors (optional for visual output indication)

Introduction:

Logic gates are fundamental components in digital electronics, performing specific logical functions on binary inputs. This experiment focuses on five basic logic gates: AND, OR, NOT, NOR, and NAND. Gaining a clear understanding of their functions and validating their truth tables is essential for anyone involved in digital logic design. The AND gate delivers a high output only when all its inputs are high; the OR gate gives a high output if any input is high; meanwhile, the NOT gate, or inverter, provides the opposite signal of its input. In contrast, the NAND gate negates the AND operation, while the NOR gate negates the OR operation. This hands-on experiment is designed to implement these gates and confirm their performance against established truth tables.

Procedure:

1. **Prepare the Breadboard:** Start by connecting the 5V power supply to the breadboard's power rails, ensuring the correct polarity.

2. **Insert Logic Gate ICs:** Place the integrated circuits (ICs) for each type of gate (7408, 7432, 7404, 7402, 7400) on the breadboard, making sure they are positioned correctly. Consult the IC datasheets for pin assignments for VCC, GND, inputs, and outputs.
3. **Connect Power and Ground:** Attach the VCC pin from each IC to the power rail and the GND pin to the ground rail.
4. **Wire the Inputs:** For every gate, connect the input pins to wires that can toggle between the power rail (logic HIGH or 1) and GND (logic LOW or 0).
5. **Connect the Output:**
   - → Connect the output pin of each gate to a multimeter to measure output voltage.
   - → Optionally, link the output pin of each gate to an LED (with a current-limiting resistor) to visually indicate the output state (LED ON = HIGH, LED OFF = LOW).
6. **Test Input Combinations:**
   - → Systematically apply all possible input combinations for each gate.
   - → For the NOT gate, test both input states (0 and 1).
   - → For the 2-input gates (AND, OR, NAND, NOR), evaluate all four combinations: (0,0), (0,1), (1,0), and (1,1).
7. **Record Observations:** Document the output voltage (or LED status) for each input combination in a table, and compare the observed results with the expected values from the truth tables.

Logic Expression and Truth Table:

1. AND Gate

- **Logic Expression:** $Y = A \cdot B$
- **Function:** The AND gate performs logical conjunction. Its output (Y) is HIGH (1) only when *all* of its inputs (A, B) are HIGH (1). If any input is LOW (0), the output is LOW (0). This reflects the Boolean AND operation, requiring all conditions to be true for the result to be true.

Truth Table:

A	B	Y
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

In Simple Terms: Think of it like a requirement: both A *AND* B must be 'on' for Y to be 'on'. Otherwise, Y is off.

2. OR Gate

- **Logic Expression:** Y = A + B
- **Function:** The OR gate performs logical disjunction. Its output (Y) is HIGH (1) if *at least one* of its inputs (A, B) is HIGH (1). The output is LOW (0) only when *all* inputs are LOW (0). It can be thought of as finding the *maximum* between two binary digits.

Truth Table:

A	B	Y
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

In Simple Terms: If A *OR* B (or both) is 'on', then Y is 'on'. Otherwise, Y is off.

3. NOT Gate

- **Logic Expression:** Y = ¬A or Y = A'
- **Function:** The NOT gate, also known as an inverter, performs logical negation. It has a single input (A) and reverses the logic state. If the input is HIGH (1), the output (Y) is LOW (0), and vice versa.

Truth Table:

A	Y
0	1
1	0

In Simple Terms: The NOT gate simply "flips" the input. If the input is 'on', the output is 'off', and if the input is 'off', the output is 'on'.

4. NOR Gate

- **Logic Expression:** Y = ¬(A + B) or Y = A NOR B
- **Function:** The NOR gate is a universal logic gate that produces a HIGH output (1) only when all its inputs are LOW (0). It combines the functions of an OR gate followed by a NOT gate, effectively inverting the output of the OR operation. If either input is HIGH (1), the output will be LOW (0).

Truth Table:

A	B	Y
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**0**

In Simple Terms: The NOR gate outputs 'on' only when both inputs are 'off'. If either input is 'on', the output is 'off'. This gate is functionally complete, meaning it can be used to create any other logic gate, including AND, OR, and NOT gates, making it highly versatile in digital circuit design.

5. NAND Gate

- **Logic Expression:** Y = ¬(A · B)
- **Function:** The NAND gate (short for NOT-AND) is a universal logic gate. It produces a LOW (0) output only if all its inputs are HIGH (1); otherwise, the output is HIGH (1). It's the complement of the AND gate. NAND gates are functionally complete, meaning any other logic gate can be created from them.

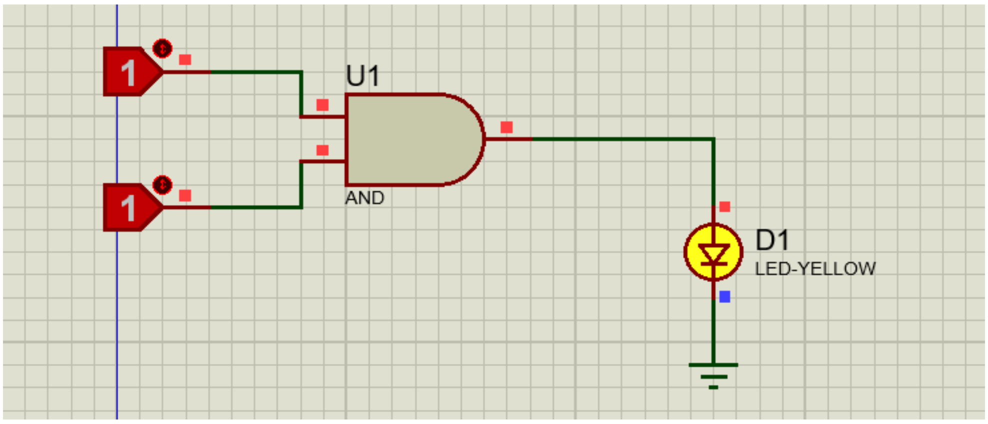
Truth Table:

A	B	Y
0	0	**1**
0	1	**1**
1	0	**1**
1	1	**0**

In Simple Terms: The NAND gate gives an output of "on" (1) unless all the inputs are "on" (1), in which case it gives an output of "off" (0). It is widely used in digital circuits because of its functional completeness.

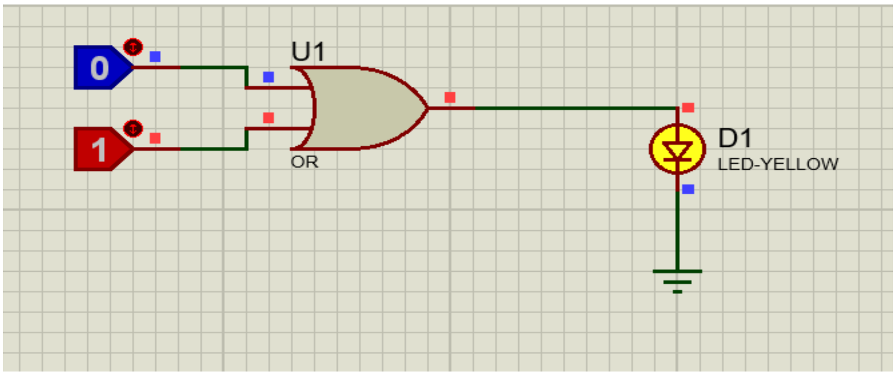
Simulated Results:

1. **AND Gate**

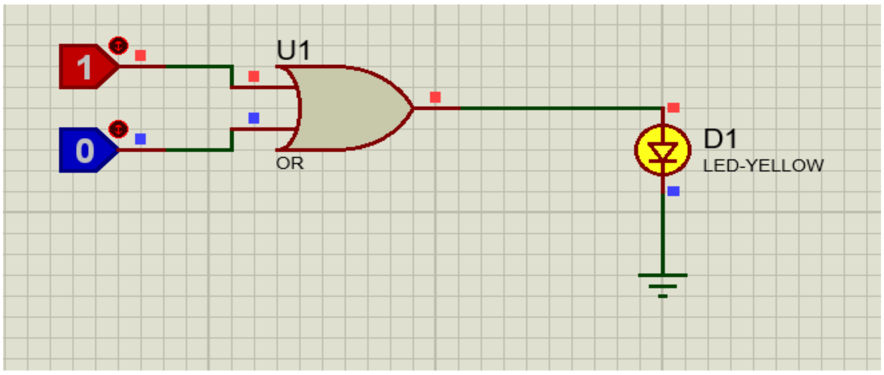


2. **OR Gate**

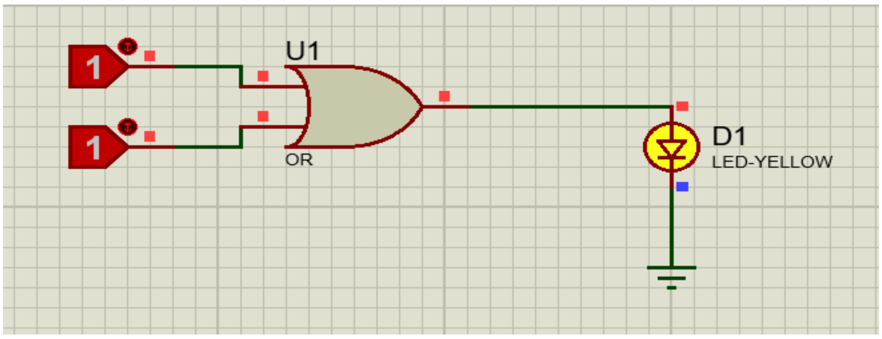
For 0 and 1 input



For 1 and 0 input

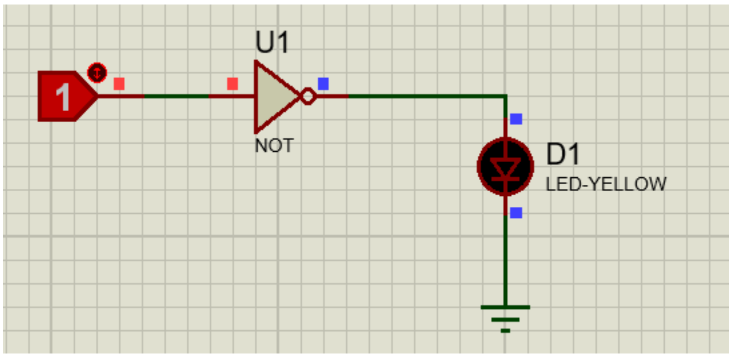


For 1 and 1 input

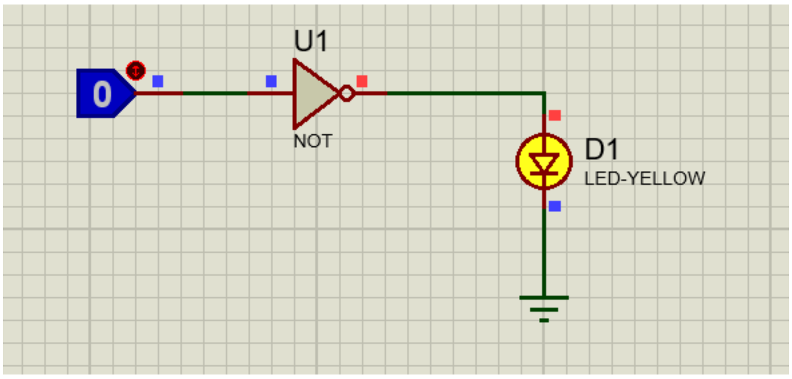


3. NOT Gate

For 1 input

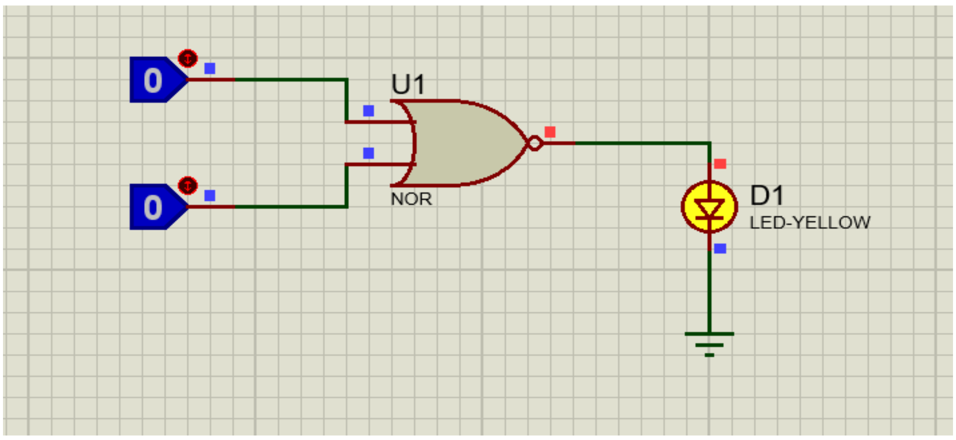


For 0 input



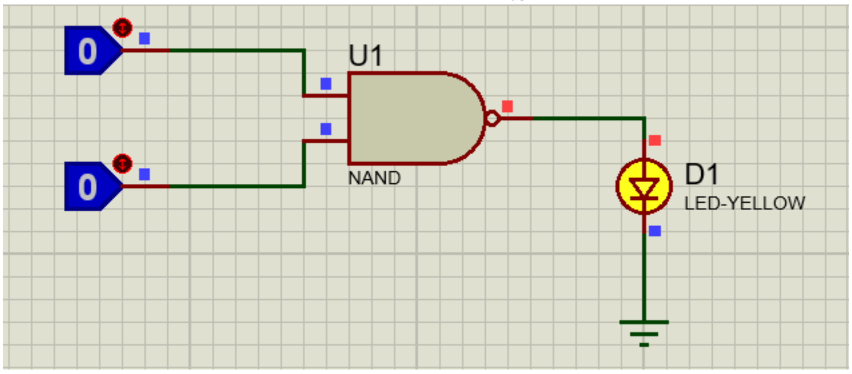
4. NOR Gate

For Both 0 inputs

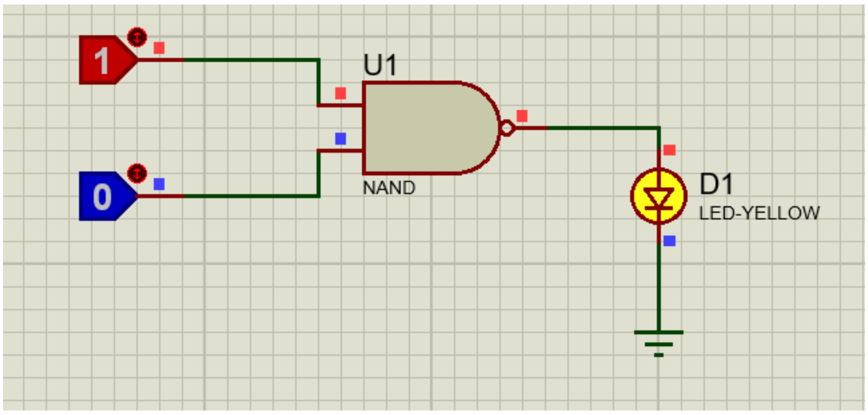


5. **NAND Gate**

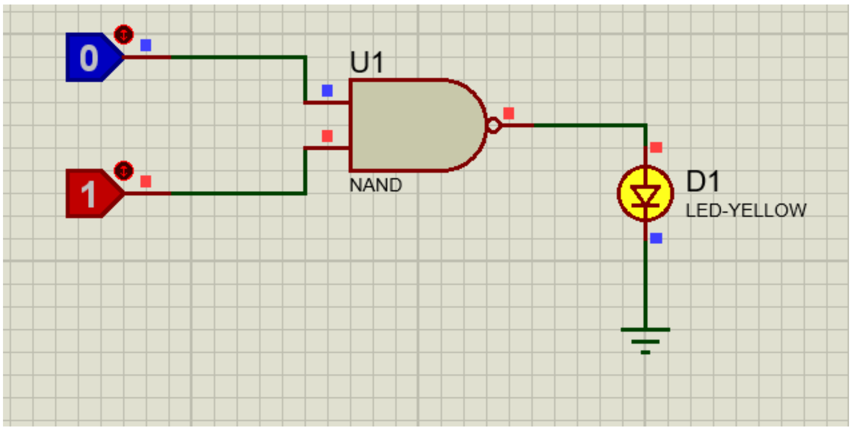
For both 0 inputs



For 1 and 0 input



For 0 and 1 input



Conclusions:

This experiment successfully demonstrated how the AND, OR, NOT, NOR, and NAND logic gates work. By setting up basic circuits on a breadboard and testing all combinations of inputs, we confirmed that the outputs of the gates aligned with the expected truth table values. This activity offered a comprehensive insight into these crucial components of digital circuits. Future iterations of this experiment could involve building more complex circuits, such as adders or flip-flops, or utilizing logic gate simulation software to explore more intricate designs. The concepts illustrated in this experiment hold wide-ranging applications in digital systems, including microprocessors, memory devices, and control circuits, emphasizing their importance in modern technology.

THANK YOU FOR YOUR TIME!

THE END